Learning RabbitMQ

Ryan Ermita
Jul 26, 2019 · 10 min read

We are on the process on converting our software architecture to Microservices and one way to communicate on each of these microservices is by using a message broker. So, a few weeks ago, we did an R&D about RabbitMQ. Our goal was to understand what is RabbitMQ, how does it work, and finally figure out how can we implement RabbitMQ on our microservices. We're moving to Microservices architecture to make our system more scalable, among other reasons. On our previous architecture, we're using Redis as a message broker. Redis is easy to set up, use and deploy but based on what I read, RabbitMQ is the way to go for more scalable software. Other than the scalability issues, Redis has these following problems as a message broker:

- Does not support TLS by default. In Redis, securing messages and connection can be done by tunneling strategies. Redis recommend Spiped.
- Only support basic message queuing and routing.
- High percentage of message loss when Redis, publisher or consumer crashes.
- High latency in dealing with large messages. Redis is better suited for small messages.
- Redis was built with different intention, in-memory key-value database, and not for being a message broker.

We're hoping that by implementing RabbitMQ on our Microservices architecture we could solved and prevent these problems.

. . .

What is RabbitMQ

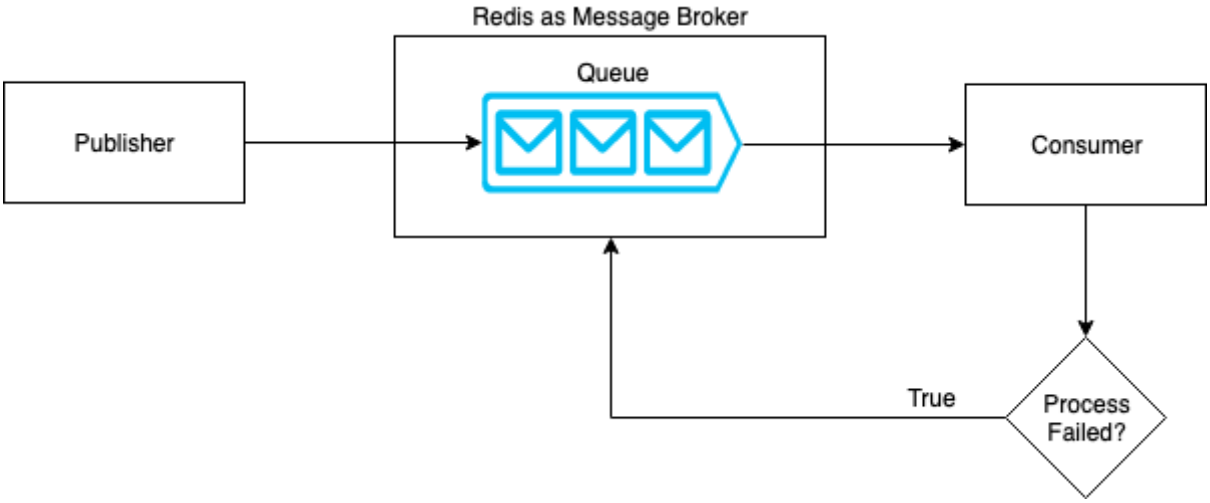
RabbitMQ is a message broker that originally implements the Advance Message Queuing Protocol (AMQP), but now it supports different messaging protocol via plugins. AMQP is an open standard for passing business messages between applications or organizations. AMQP standards was designed with the following main characteristics: Security, Reliability, Interoperability, Standard, Open. So how does RabbitMQ implement this characteristics:

- Security —support authentication, authorization, LDAP, and TLS via RabbitMQ plugins.
- Reliability — confirms the message was successfully delivered to the message broker and confirms that the message was successfully processed by the consumer. RabbitMQ also have a builtin clustering feature that results to high availability, and scalability. There's also an option to make your data persistent so the message wont be lost in case the broker quits or crashes.
- Interoperability — message is transfer as stream of bytes so any clients can operate on it. RabbitMQ supports a lot of client libraries and and dev tools, in different programming languages.
- Open and Standard — aside from following the open standards of AMQP, RabbitMQ is open source and anyone can contribute to make it better.

. . .

RabbitMQ Architecture

First, lets see how we implement Redis as a message broker, it follows this process:



1. An application publish a message to the message broker, which is in this case, Redis. The message was directly pushed to the queue.

2. The message is stored in a queue waiting to be consumed by a consumer from the same or different application.

3. A consumer consumes the message from the queue. The moment the message was consumed, it is deleted from the queue. Take note that on this part, the consumers was the one whose retrieving the message from the queue.

4. If the consumer fails to process the message, the consumer will push the message to the queue and the process will repeat from step 2.

This process is very simple and straightforward but, it is fragile, not flexible, and hard to scale. It wont be able to handle these cases:

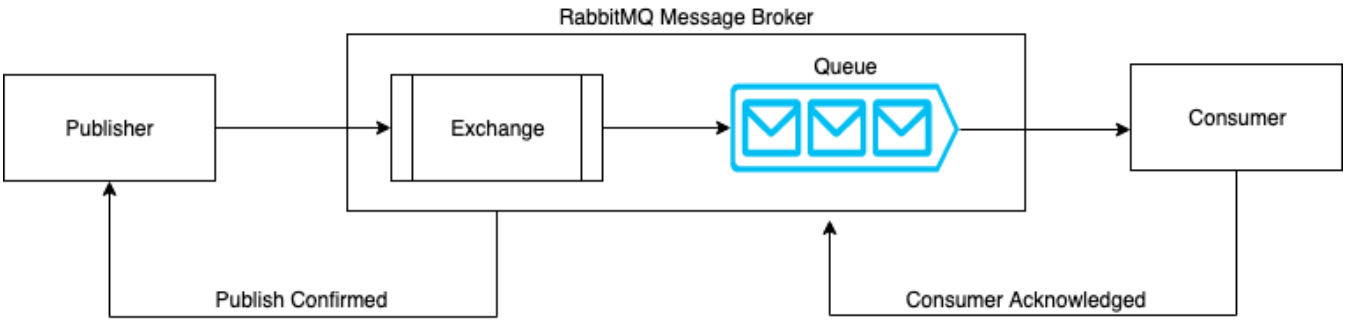
- How can I make sure the message was successfully published to the message broker?

- What if Redis crashes? There's a high possibility that the messages on route to the queue will be gone and there's no available message broker to handle the incoming and outgoing messages.

- What if the consumer crashes the time it consumed a message from the queue. The message will not be re-queued.

- What if I want to publish the message to more than one queue or to the queues that met a set of criteria? For this to be possible, We need to manually modify our code base.

Those cases above can be easily solved by RabbitMQ and its not that hard to implement. But first, lets see how RabbitMQ message broker works:



1. The application publish a message to the message broker, in this case, RabbitMQ. The message was pushed to an Exchange instead of a queue.

2. The Exchange will route the message to the queue or queues that is bound to the Exchange.

3. The RabbitMQ message broker can notify the publisher if the message was successfully routed to the queue or queues and if it fails to route the message, the Exchange can notify the publisher that the message was unable to route. On this failed scenario, the publisher has an option to republish the message or not.

4. The message is stored in a queue waiting for an active consumer, if there are any active consumer, the message broker delivers the message from the queue to the active consumer.

5. A consumer consumes the message sent by the message broker from the queue. The consumer can automatically or manually send an acknowledgment to message broker that the message was successfully processed and the message can be safely remove from the queue.

This is the high level approach and architecture of RabbitMQ message broker. Compared with Redis as message broker. RabbitMQ have an additional component, the Exchange that routes the message to the queue or queues. Also, RabbitMQ provides a mechanism that is essential to data safety. We can guarantee that the message was successfully routed to the queue or queues else we have an option to republish the message, and we can guarantee that the message was successfully processed by the consumer else we can re-queue the message so it can be consumed by other consumer. By understanding this approach and architecture we can conclude that RabbitMQ is not just simple but also a robust message broker.

. . .

Code Examples

These code examples is originally came from the RabbitMQ tutorial, I just did some modification so we can create a robust application using RabbitMQ. Also, these codes are written using Python Pika RabbitMQ Client. We will dissect this codes line by line to have better understanding how RabbitMQ works.

Publisher example:

```
1  import pika
2  import sys
3
4  connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
5  channel = connection.channel()
6
7  channel.exchange_declare(exchange='direct_exchange', exchange_type='direct')
8  channel.queue_declare(queue='direct_queue', durable=True)
9  channel.queue_bind(exchange='direct_exchange', queue="direct_queue", routing_key="direct.routin
10
11 message = " ".join(sys.argv[1:]) or "Hello World!"
12
13 channel.confirm_delivery()
14 try:
15     channel.basic_publish(exchange='direct_exchange', routing_key='direct.routing.key',
16                           body=message, properties=pika.BasicProperties(delivery_mode=2)
17                           )
18
19     print("Sent %r" % message)
20 except pika.exceptions.UnroutableError:
21     print("Failed to send message %r" % message)
22 connection.close()
```

publisher.py hosted with ♡ by GitHub view raw

```
connection =
pika.BlockingConnection(pika.ConnectionParameters('localhost'))
```

After importing the required packages on line1–2. Line 4, We create a RabbitMQ connection instance, this connection uses TCP as protocol. TCP protocol deals only with packets (bits of data) and enable the connection between two host so that they can exchange data. Also TCP guarantee that the message are delivered in order in which they were sent.

```
channel = connection.channel()
```

Line 5, we create a channel, all the client operations happens on a channel. We can have more than one channel in one connection. The reason behind this is that: *Some*

applications need multiple logical connections to the broker. However, it is undesirable to keep many TCP connections open at the same time because doing so consumes system resources and makes it more difficult to configure firewalls. So, channels can be thought of as "lightweight connections that share a single TCP connection".

```
channel.exchange_declare(exchange='direct_exchange',
exchange_type='direct')
```

Line 7, we create an Exchange. As what we've discussed earlier, The responsibility of the Exchange is to route the messages to the queue or queues. Exchange knows where to route the messages based on the specified routing key. We declare our Exchange with two parameters: `exchange` — the name of the exchange, and the `exchange_type` — the type of the exchange controls how the message will be routed. There are four types of exchange:

- direct exchange — delivers messages to queues based on the message routing key.
- fanout exchange — routes messages to all of the queues that are bound to it and the routing key is ignored.
- topic exchanges — route messages to one or many queues based on matching between a message routing key and the pattern that was used to bind a queue to an exchange. Routing keys follows this pattern *<word>.<word>.<n-word>*, and to find a match we use * (star) to substitute for exactly one word and # (hash) to substitute for zero or more words.
- headers exchange — is designed for routing on multiple attributes that are more easily expressed as message headers than a routing key.

```
channel.queue_declare(queue='direct_queue', durable=True)
```

Line 8, We create a queue with `queue` — the queue name and `durable` as parameter. When RabbitMQ quits or crashes it will forget the queues and messages unless you tell it not to. By setting our queue as `durable`, we can make sure that even if RabbitMQ quits or crashes, our queues wont be deleted.

```
channel.queue_bind(exchange='direct_exchange', queue="direct_queue",
routing_key="direct.routing.key")
```

Line 9, we bind our queue to an exchange and specified the routing key. As a result, Exchange now knows where to route the messages based on the specified routing key and the type of the exchange.

```
channel.confirm_delivery()
```

Line 13, we enable publish confirms, by doing so the message broker will raise an error if it fails to route our messages to our queue or queues. Take note that by enabling publish confirms, it adds a little overhead as the message broker needs to confirm the message delivery to the publisher.

```
try:
    channel.basic_publish(exchange='direct_exchange',
routing_key='direct.routing.key',
                          body=message,
properties=pika.BasicProperties(delivery_mode=2)
                          )

    print("Sent %r" % message)
except pika.exceptions.UnroutableError:
    print("Failed to send message %r" % message)
```

Line 14–21, we publish a message to the queue. Based on the parameters we tell the publisher to publish our message (body parameter) to an exchange named `direct_exchanged` with the routing key `direct.routing.key` .These parameters are self explanatory except the properties parameter. With additional properties, we tell the publisher to deliver our message using `delivery_mode=2` meaning we want to make our message persistent. Just like with queues, messages are non-persistent unless we told RabbitMQ to make it persistent. Non-persistent queues and messages will be deleted in case RabbitMQ quits or crashes, by making the queues and messages persistent we can make sure that the queues and message will survive in case RabbitMQ quits or crashes. and on line 20, we catch an exception when the message broker fails to route our message to the queue or queues. This gives us an option if we want to republish the message or drop it.

```
connection.close()
```

Line 22, we're closing the connection. It is not a good practice to open and close connections and channels every time we publish a message. Connections are long lived and it takes resources to keep opening and closing them. I just include this line for the example purposes on how to close the connection.

Consumer Example:

```
import pika
import time

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()

channel.exchange_declare(exchange='direct_exchange', exchange_type='direct')
channel.queue_declare(queue='direct_queue', durable=True)
channel.queue_bind(exchange='direct_exchange', queue="direct_queue", routing_key="direct.routin


def callback(ch, method, properties, body):
        print("Received %r" % body)
        time.sleep(body.count(b'.'))
        print("Done")
        ch.basic_ack(delivery_tag=method.delivery_tag)


channel.basic_qos(prefetch_count=1)
channel.basic_consume(callback, queue='direct_queue')

print('Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

consumer.py hosted with ♡ by GitHub view raw

```
connection =
pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

Line 1–9 have the same explanation with the publisher app. For Line 4–5, Its understandable that we need to create a connection and channel so we can connect to

the message broker.

```
channel.exchange_declare(exchange='direct_exchange',
exchange_type='direct')
channel.queue_declare(queue='direct_queue', durable=True)
channel.queue_bind(exchange='direct_exchange', queue="direct_queue",
routing_key="direct.routing.key")
```

But for line 7–9, it doesn't make sense, because we already created and configure the queue in the publisher app. The reason behind this redundancy is that we need to make sure that the queue exist or else RabbitMQ will just drop the message. So technically, declaring and configuring the queue on the publisher and consumer app is considered a good practice in RabbitMQ. Declaring a queue with the same name and properties is idempotent, meaning we can run it multiple times but only one queue will be created. But take note that if we create a queue with the same name with different properties, RabbitMQ will raise an error.

```
def callback(ch, method, properties, body):
        print("Received %r" % body)
        time.sleep(body.count(b'.'))
        print("Done")
        ch.basic_ack(delivery_tag=method.delivery_tag)
```

Line 12–16 is our callback function, meaning this will be triggered once we consume a message from the queue. This callback function requires four parameter the:

- `ch` — the channel instance.
- `method` — include the details how the message is delivered (e.g `routing_key`, `exchange`, and `delivery_tag`)
- `properties` — the properties we set on the publisher (e.g `delivery_mode`)
- `body` — the message we consumed from the queue. it is in bytes datatype.

```
ch.basic_ack(delivery_tag=method.delivery_tag)
```

Line 16, we acknowledge that the message was successfully processed by the consumer and it is now safe to delete it from the queue. By default, the acknowledgment happens automatically. This means that once the message was consumed by the consumer the message in the queue will be deleted even though the consumer is not done processing the message. This mode is often referred to as “fire-and-forget”. Unlike with manual acknowledgement model, if consumers’s TCP connection or channel is closed before successful delivery, the message sent by the server will be lost. Therefore, automatic message acknowledgement should be considered unsafe and not suitable for all workloads.

```
channel.basic_qos(prefetch_count=1)
```

Line 19, we set the qos (Quality of Service) with `prefetch=1`, to make sure only one message will be consumed and the RabbitMQ wont push any message to the consumer until the current message was acknowledged. If we don’t set any qos `prefetch` the consumer will accept as much number of messages it can handle and this can cause bottleneck as we can have as much number of inflight and unacknowledged messages on the consumer that supposedly can handle by another consumer instance.

```
channel.basic_consume(callback, queue='direct_queue')
```

Line 20, we set the consumer to consume from the `direct_queue` and to set our `callback()` as consumer callback. So every time the consumer consumes a message from the queue this `callback()` will be automatically triggered.

```
channel.start_consuming()
```

And lastly, on Line 23, we trigger an infinite loop that waits for a message and trigger our `callback()`.

Now by dissecting our code examples line by line, we have a better grasp and understanding on how RabbitMQ works and how we can implement this robust message broker on our applications whenever it is applicable.

Happy Coding!

. . .

Resources

- RabbitMQ in 5 minutes by Bernhard Wenzel Training
- Reliable Messaging With RabbitMQ — Part 1 by JimOnDemand
- RabbitMQ Exchange Types and its use cases with Examples by Tech WatchDog
- High availability and failover in RabbitMQ by Tech WatchDog
- Redis vs RabbitMq as a message broker by Vishnu Kiran K V
- AMQP is the Internet Protocol for Business Messaging
- AMQP 0–9–1 Model Explained
- Consumer Acknowledgements and Publisher Confirms
- RabbitMQ Channels
- RabbitMQ Consumers
- RabbitMQ Basic Python Tutorial
- RabbitMQ Basic Python Tutorial — Work Queues
- RabbitMQ Basic Python Tutorial — Publisher/Subscriber
- RabbitMQ Basic Python Tutorial — Routing
- RabbitMQ Basic Python Tutorial — Topics
- RabbitMQ Libraries and Devtools

Message Broker Rabbitmq Software Engineering

Medium About Help Legal